

CS231n 4-6

김동원

목차

Backpropagation

Fully connected Layer

Convolutional Layer

Data Preprocessing

Weight Initialization

Batch Normalization

Hyperparameter Optimization

Backpropagation

계산 예시

학습을 위한 가중치를 효율적으로 업데이트 하기위한 방법

Backpropagation: a simple example

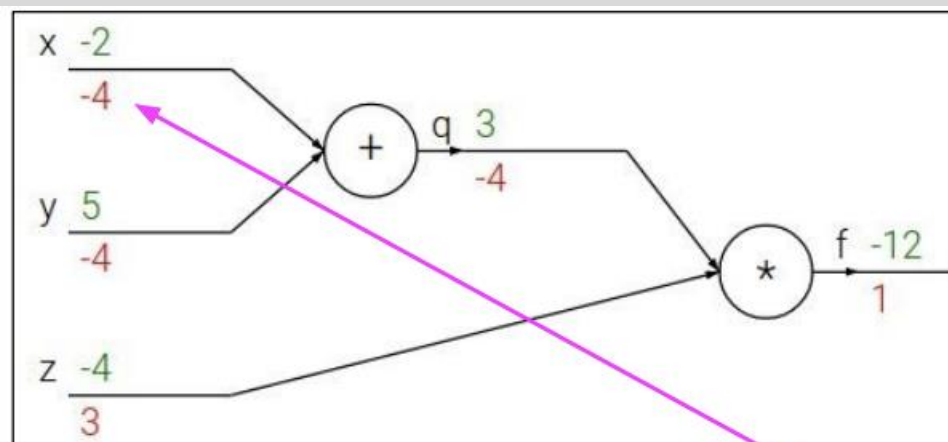
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



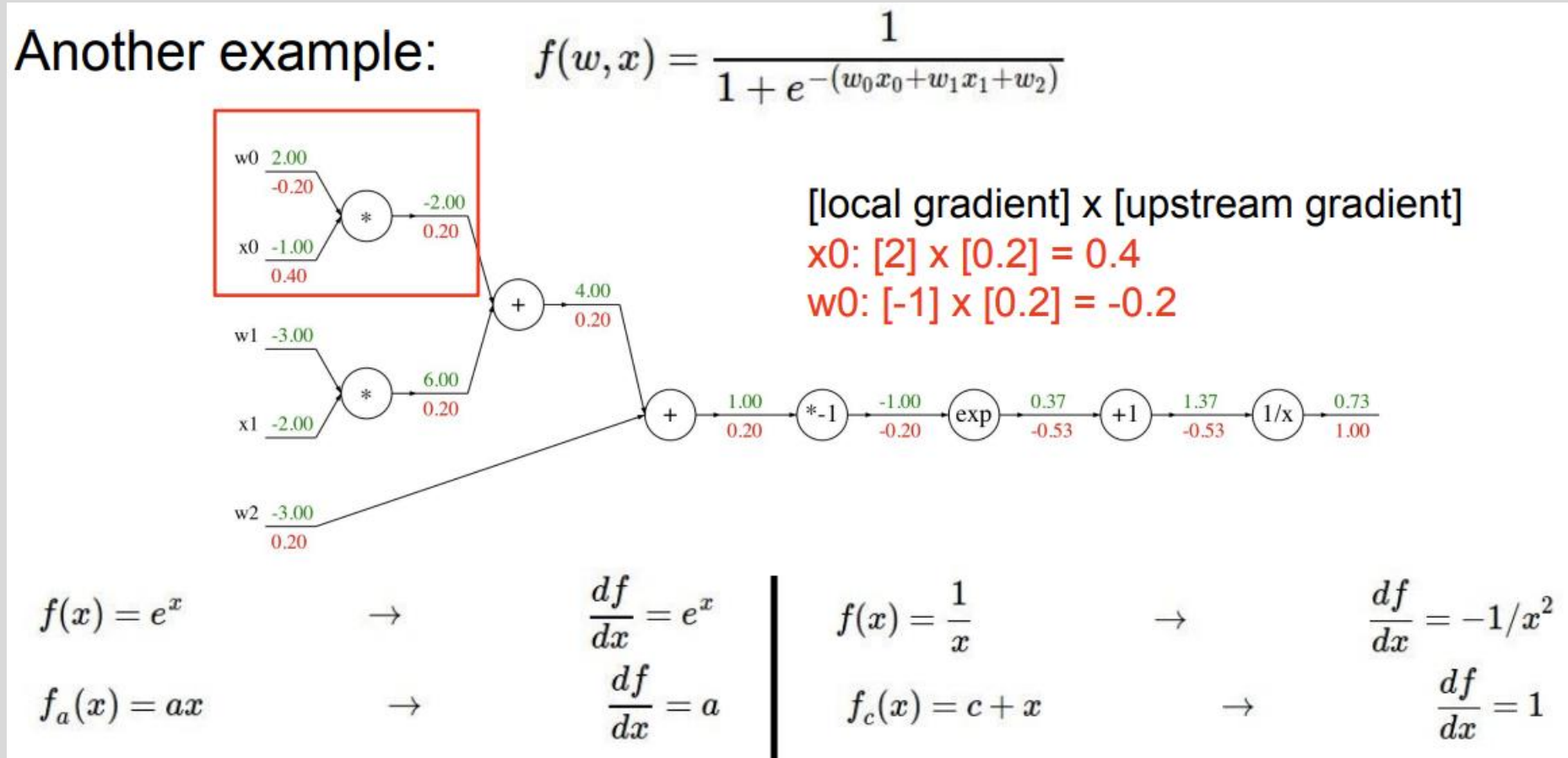
$$\frac{\partial f}{\partial x}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Backpropagation

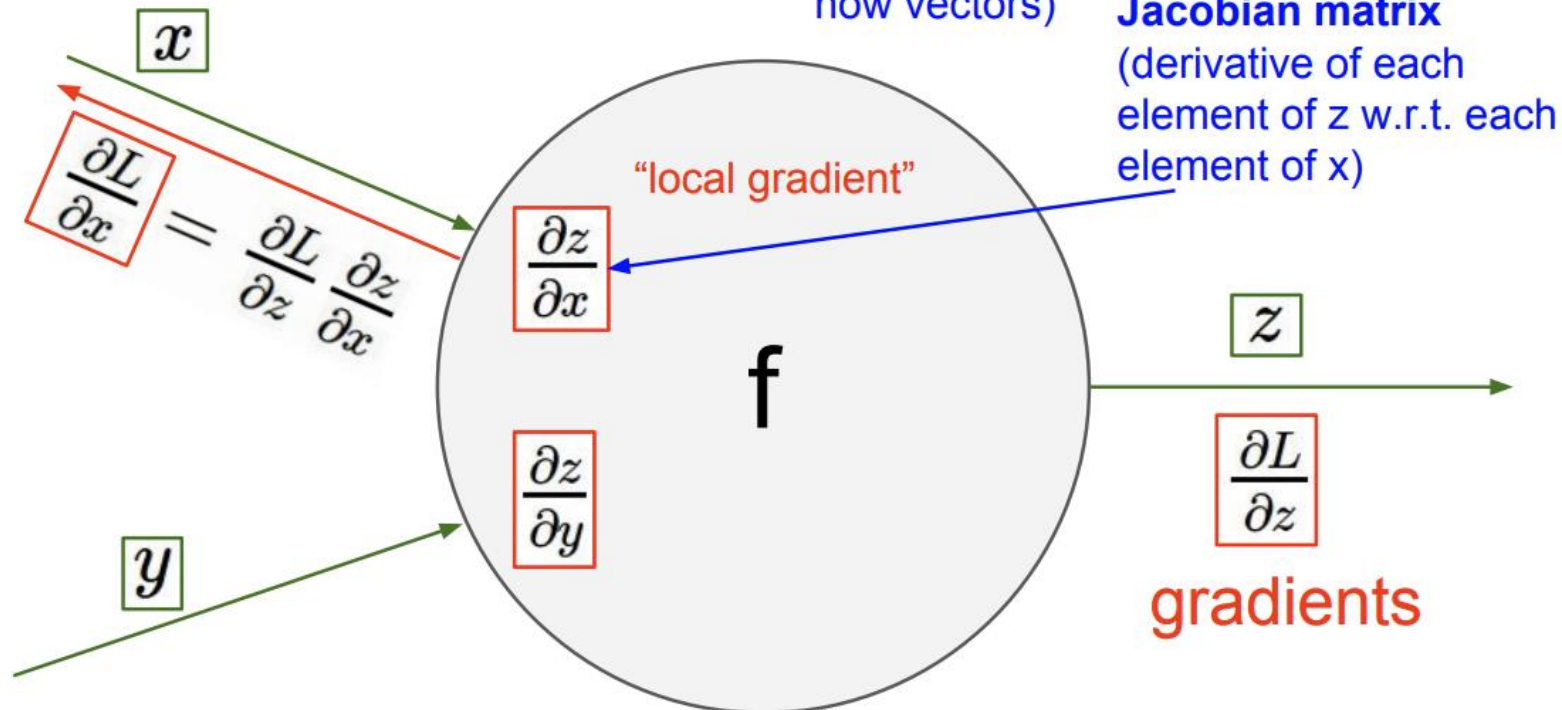
계산 예시2



Backpropagation

벡터 입력의 계산

Gradients for vectorized code

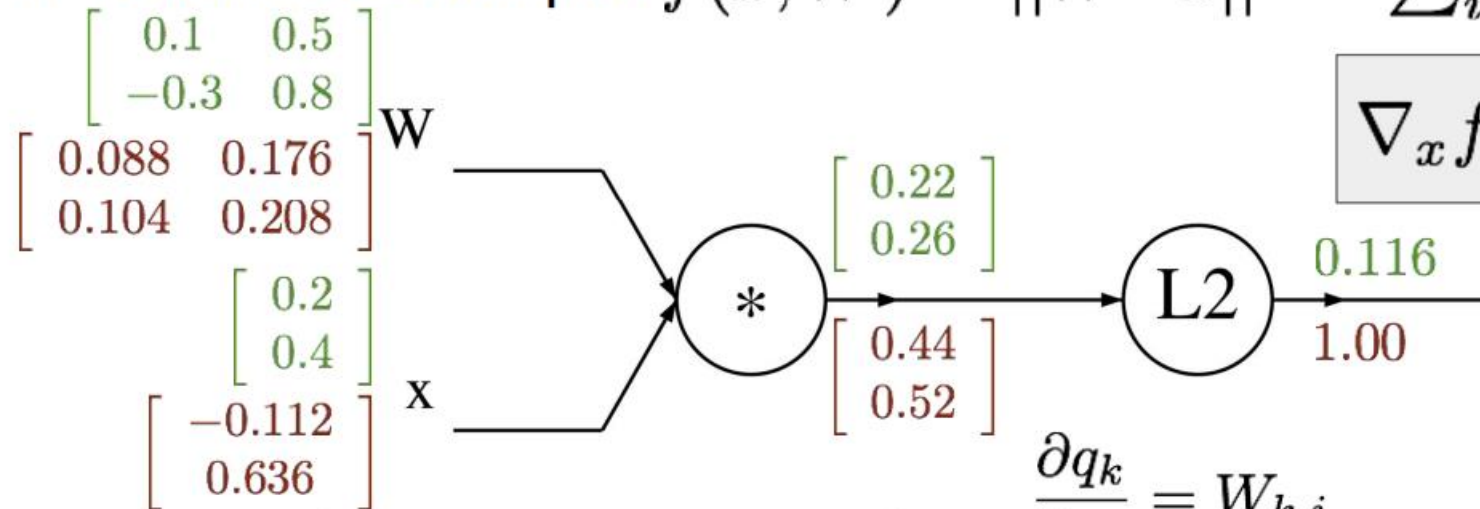


$$J = \begin{bmatrix} \frac{\partial F_1}{\partial x_1} & \cdots & \frac{\partial F_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial F_m}{\partial x_1} & \cdots & \frac{\partial F_m}{\partial x_n} \end{bmatrix}$$

Backpropagation

벡터 계산 예시

A vectorized example: $f(x, W) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$



$$\nabla_x f = 2W^T \cdot q$$

$$\nabla_W f = 2q \cdot x^T$$

$$q = W \cdot x = \begin{pmatrix} W_{1,1}x_1 + \dots + W_{1,n}x_n \\ \vdots \\ W_{n,1}x_1 + \dots + W_{n,n}x_n \end{pmatrix}$$

$$f(q) = ||q||^2 = q_1^2 + \dots + q_n^2$$

$$\begin{aligned} \frac{\partial q_k}{\partial x_i} &= W_{k,i} \\ \frac{\partial f}{\partial x_i} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial x_i} \\ &= \sum_k 2q_k W_{k,i} \end{aligned}$$

$$\begin{aligned} \frac{\partial q_k}{\partial W_{i,j}} &= \mathbf{1}_{k=i} x_j \\ \frac{\partial f}{\partial W_{i,j}} &= \sum_k \frac{\partial f}{\partial q_k} \frac{\partial q_k}{\partial W_{i,j}} \\ &= \sum_k (2q_k) (\mathbf{1}_{k=i} x_j) \\ &= 2q_i x_j \end{aligned}$$

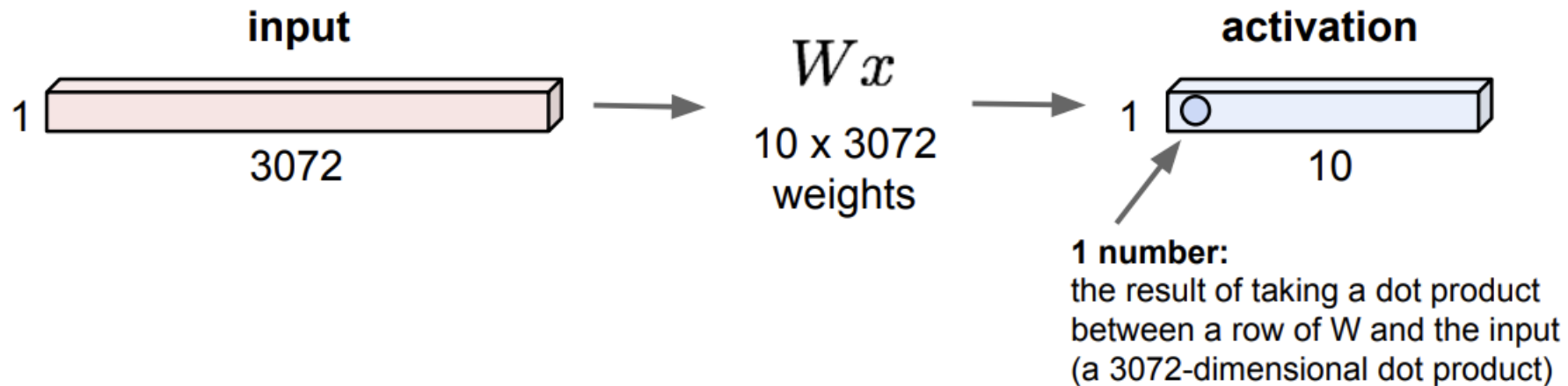
should have the same shape as variable

Fully Connected Layer

입력된 모든 데이터를 이용하여 output에 대한 연산을 할 수 있다.

Fully Connected Layer

32x32x3 image -> stretch to 3072 x 1



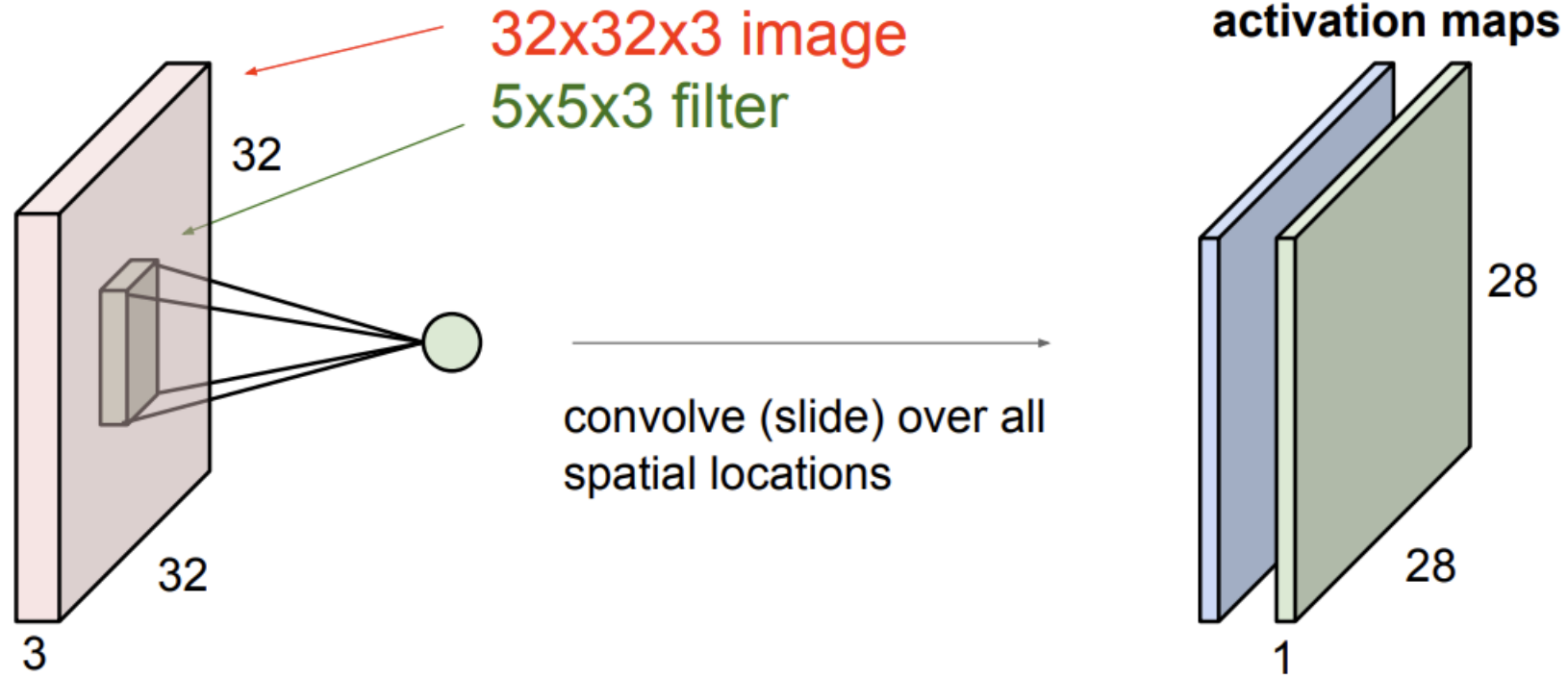
Convolution Layer

Filter과 이미지를 내적연산을 한다

Filter의 개수 만큼의 activation map이 생긴다.

Convolution Layer

consider a second, **green** filter

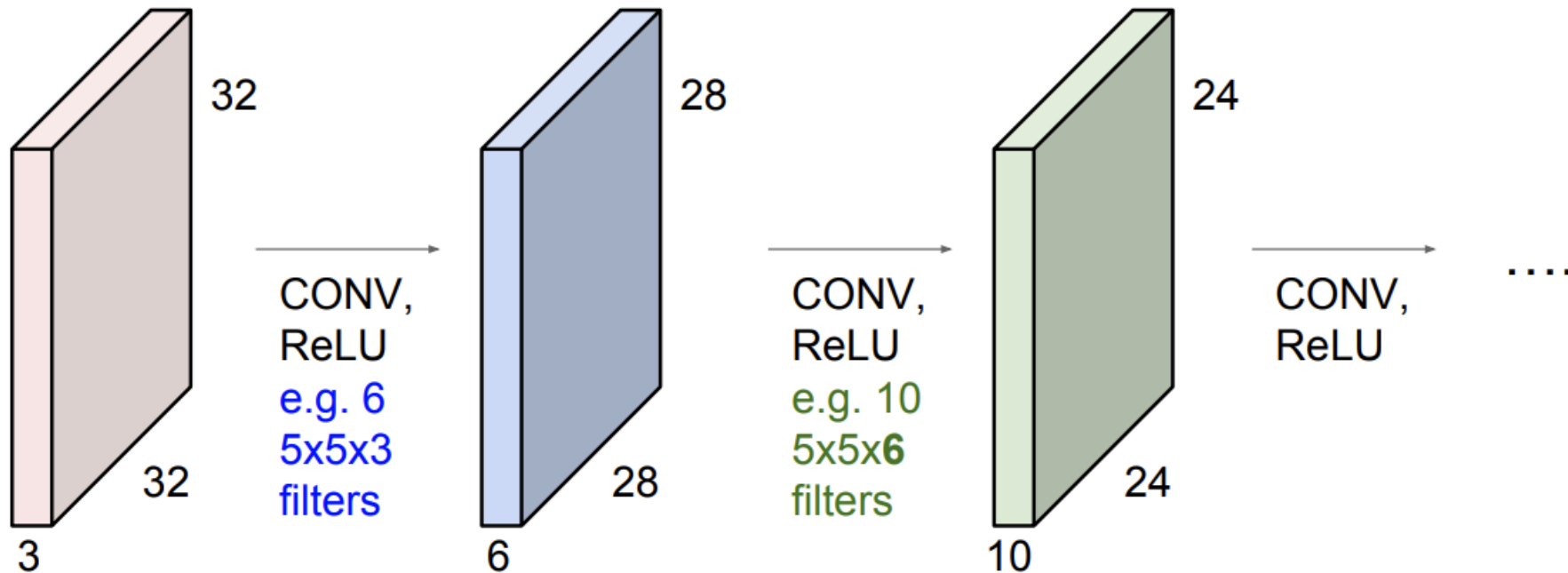


Convolution Layer

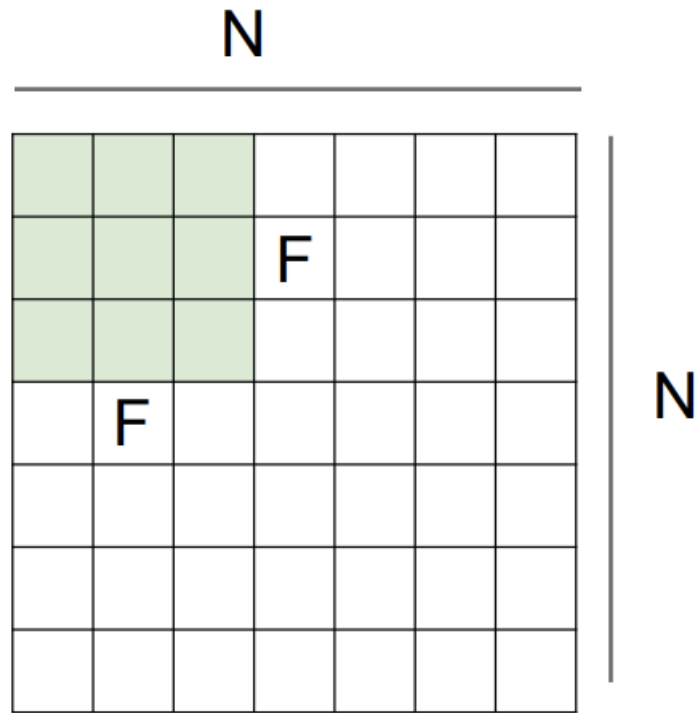
Filter과 이미지를 내적연산을 한다

Filter의 개수 만큼의 activation map이 생긴다.

Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



Convolution Layer



Output size:

$$(N - F) / \text{stride} + 1$$

e.g. $N = 7$, $F = 3$:

$$\text{stride } 1 \Rightarrow (7 - 3) / 1 + 1 = 5$$

$$\text{stride } 2 \Rightarrow (7 - 3) / 2 + 1 = 3$$

$$\text{stride } 3 \Rightarrow (7 - 3) / 3 + 1 = 2.33 \text{ :}\backslash$$

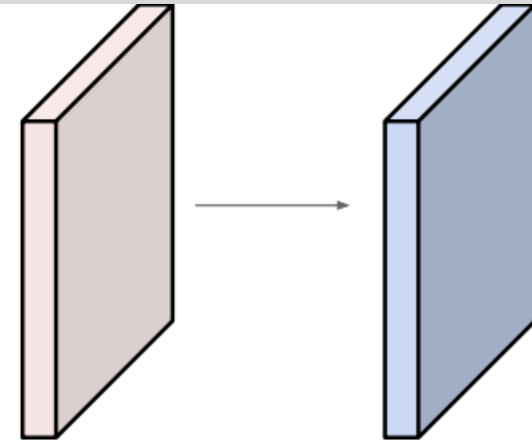
Convolution Layer

Output size계산 예시와 Layer의 parameter개수

Examples time:

Input volume: **32x32x3**

10 **5x5** filters with stride 1, pad 2



Number of parameters in this layer?

each filter has $5*5*3 + 1 = 76$ params (+1 for bias)

=> $76*10 = 760$

Data Preprocessing

0을 중심으로 옮겨 학습에 편향이 생기지 않도록 한다

TLDR: In practice for Images: center only

e.g. consider CIFAR-10 example with [32,32,3] images

- Subtract the mean image (e.g. AlexNet)
(mean image = [32,32,3] array)
- Subtract per-channel mean (e.g. VGGNet)
(mean along each channel = 3 numbers)

Not common to normalize
variance, to do PCA or
whitening

Weight Initialization

W를 작은 값으로 설정한 경우

연산과정에서 곱셈을 하며 출력이 빠르게 0에 가까워지고,
Backward과정에서는 입력이 0보다 작은 값에 곱해지며 작아지고 기울기도 0에 수렴하게 된다.

Xavier 초기화를 사용한다

Xavier 초기화는 입력과 출력의 분산을 고려하여 입력이 작으면 큰 값을 주고 큰 값이 입력되면 작은 W를 주어 위의 문제를 해결한다

Relu함수를 사용할 경우 Xavier초기화를 사용하면 Dead Relu가 발생하는 문제가 있는데 이 때문에 He초기화를 사용한다.

```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in) # layer initialization
```

```
W = np.random.randn(fan_in, fan_out) / np.sqrt(fan_in/2) # layer initialization
```

Batch Normalization

레이어의 출력을 강제로 표준화 하여 학습의 불안정성을 줄여준다
감마와 베타 값도 학습이 가능하게 만들어 학습에 최적화된 분포를 찾을 수 있다

Normalize:

$$\hat{x}^{(k)} = \frac{x^{(k)} - \mathbb{E}[x^{(k)}]}{\sqrt{\text{Var}[x^{(k)}]}}$$

And then allow the network to squash the range if it wants to:

$$y^{(k)} = \gamma^{(k)} \hat{x}^{(k)} + \beta^{(k)}$$

Note, the network can learn:

$$\gamma^{(k)} = \sqrt{\text{Var}[x^{(k)}]}$$

$$\beta^{(k)} = \mathbb{E}[x^{(k)}]$$

to recover the identity mapping.

Hyperparameter Optimization

Cross validation에서 확인할 파라미터값이나 범위를 설정하고 적은 epoch로 학습을 수행하며 val_acc를 확인하며 찾을 수 있다.

Grid search도 좋은 방법이지만 random search를 하는 것이 결과가 더 빠르고 좋다

```
max_count = 100
for count in xrange(max_count):
    reg = 10**uniform(-5, 5)
    lr = 10**uniform(-3, -6)

    trainer = ClassifierTrainer()
    model = init_two_layer_model(32*32*3, 50, 10) # input size, hidden size, number of classes
    trainer = ClassifierTrainer()
    best_model_local, stats = trainer.train(X_train, y_train, X_val, y_val,
                                           model, two_layer_net,
                                           num_epochs=5, reg=reg,
                                           update='momentum', learning_rate_decay=0.9,
                                           sample_batches = True, batch_size = 100,
                                           learning_rate=lr, verbose=False)
```

note it's best to optimize
in log space!

```
val_acc: 0.412000, lr: 1.405206e-04, reg: 4.793564e-01, (1 / 100)
val_acc: 0.214000, lr: 7.231888e-06, reg: 2.321281e-04, (2 / 100)
val_acc: 0.208000, lr: 2.119571e-06, reg: 8.011857e+01, (3 / 100)
val_acc: 0.196000, lr: 1.551131e-05, reg: 4.374936e-05, (4 / 100)
val_acc: 0.079000, lr: 1.753300e-05, reg: 1.200424e+03, (5 / 100)
val_acc: 0.223000, lr: 4.215128e-05, reg: 4.196174e+01, (6 / 100)
val_acc: 0.441000, lr: 1.750259e-04, reg: 2.110807e-04, (7 / 100)
val_acc: 0.241000, lr: 6.749231e-05, reg: 4.226413e+01, (8 / 100)
val_acc: 0.482000, lr: 4.296863e-04, reg: 6.642555e-01, (9 / 100)
val_acc: 0.079000, lr: 5.401602e-06, reg: 1.599828e+04, (10 / 100)
val_acc: 0.154000, lr: 1.618508e-06, reg: 4.925252e-01, (11 / 100)
```

nice